

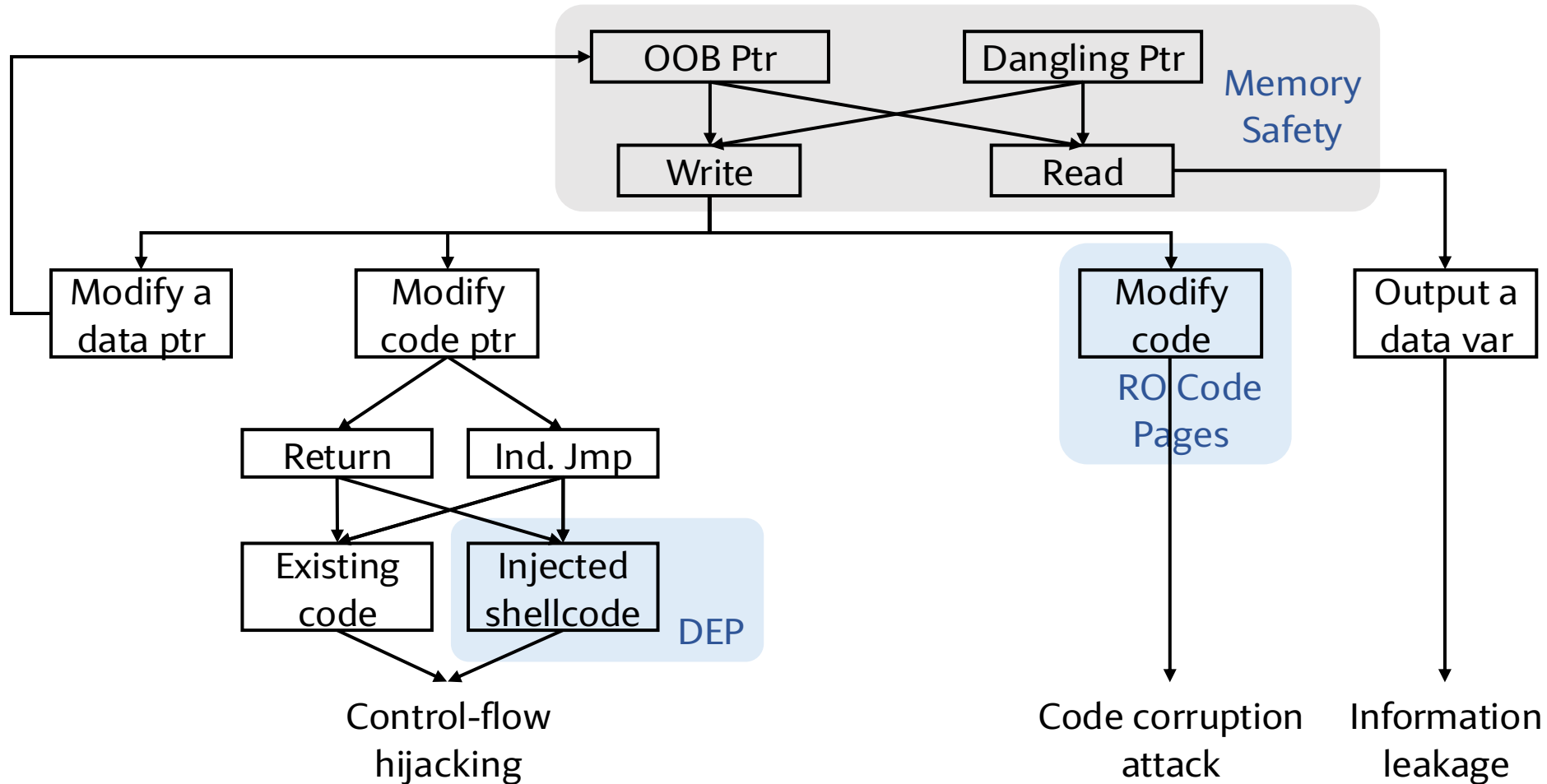
ECE 382N-Sec (FA26):

L11: Memory Safety Defense

Neil Zhao

neil.zhao@utexas.edu

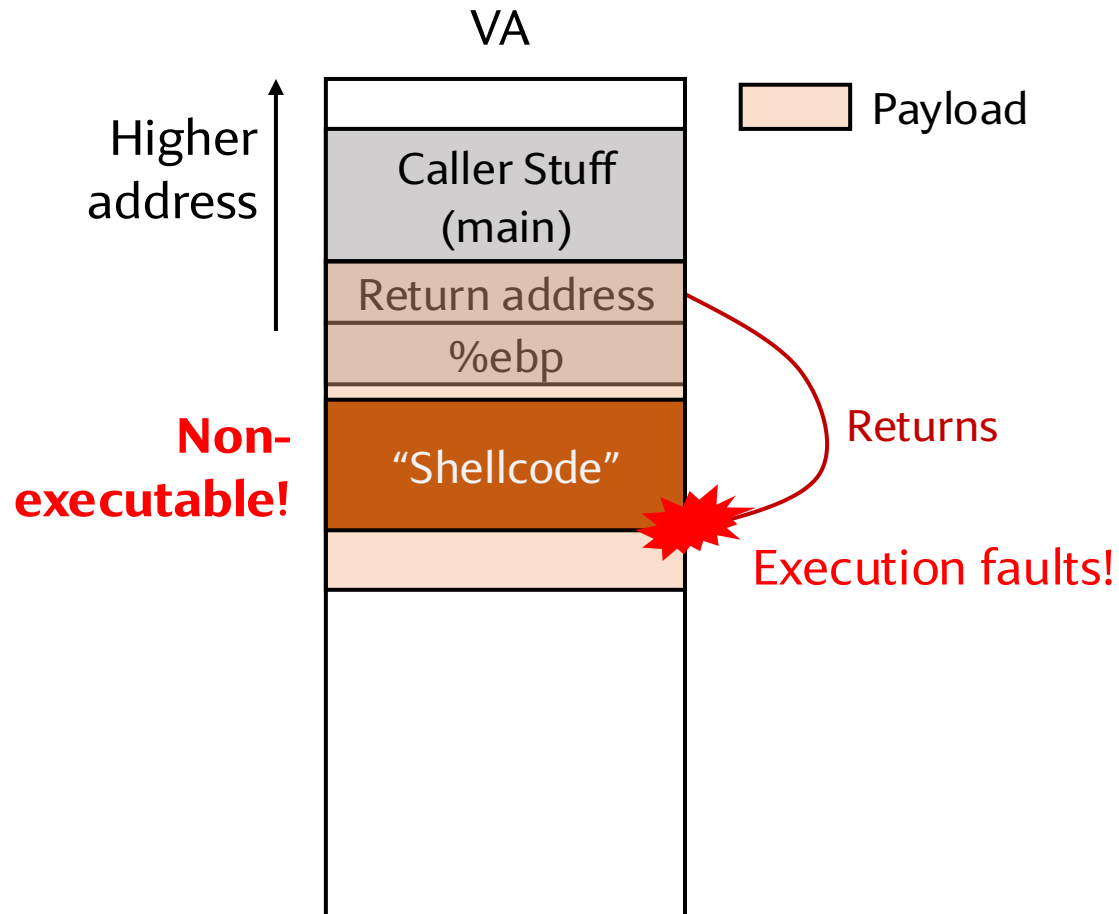
Adapted from “SoK: Eternal War in Memory”¹ (Simplified²)



¹Szekeres et al., “SoK: Eternal War in Memory,” SP’13

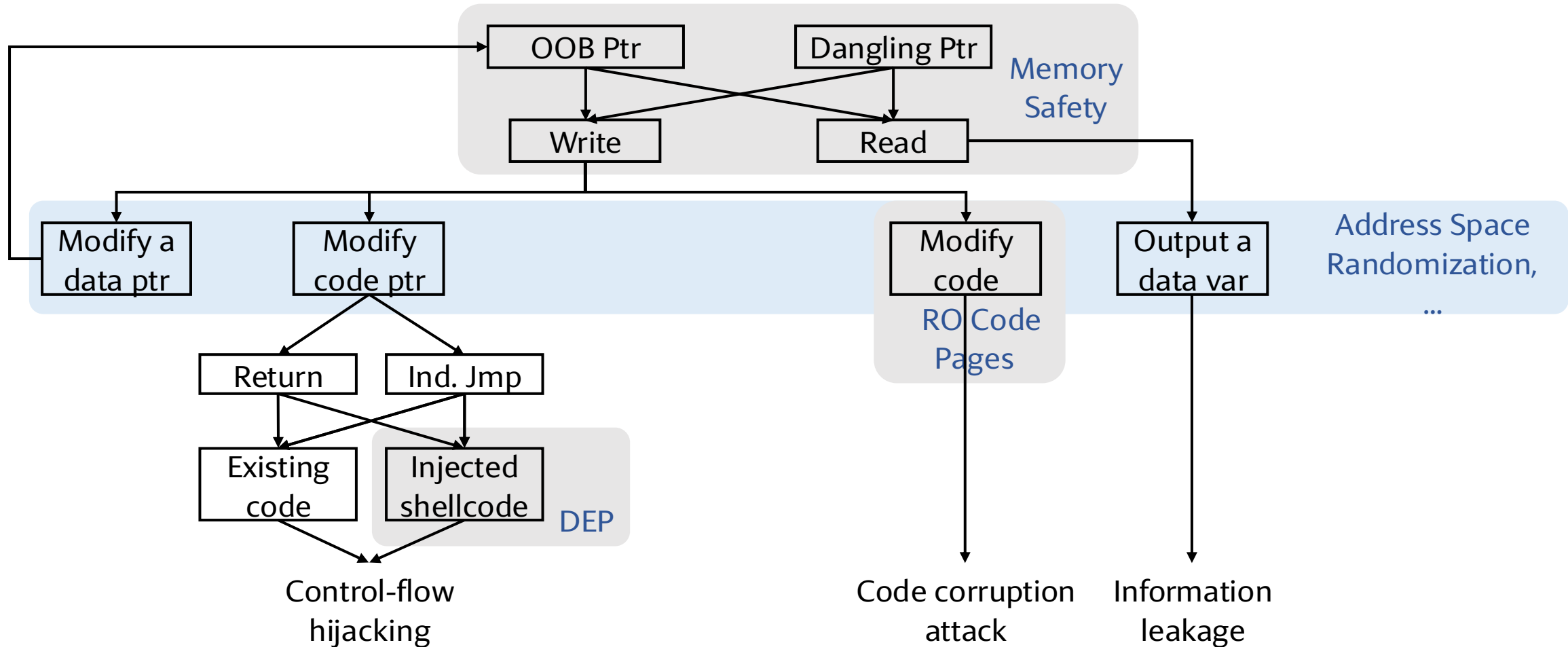
²Only a subset of defenses are listed. Some adaptations are made.

Data Execution Prevention (DEP)



Common Policy: a page is either writable or executable, but not both! (i.e., $W \oplus X$)

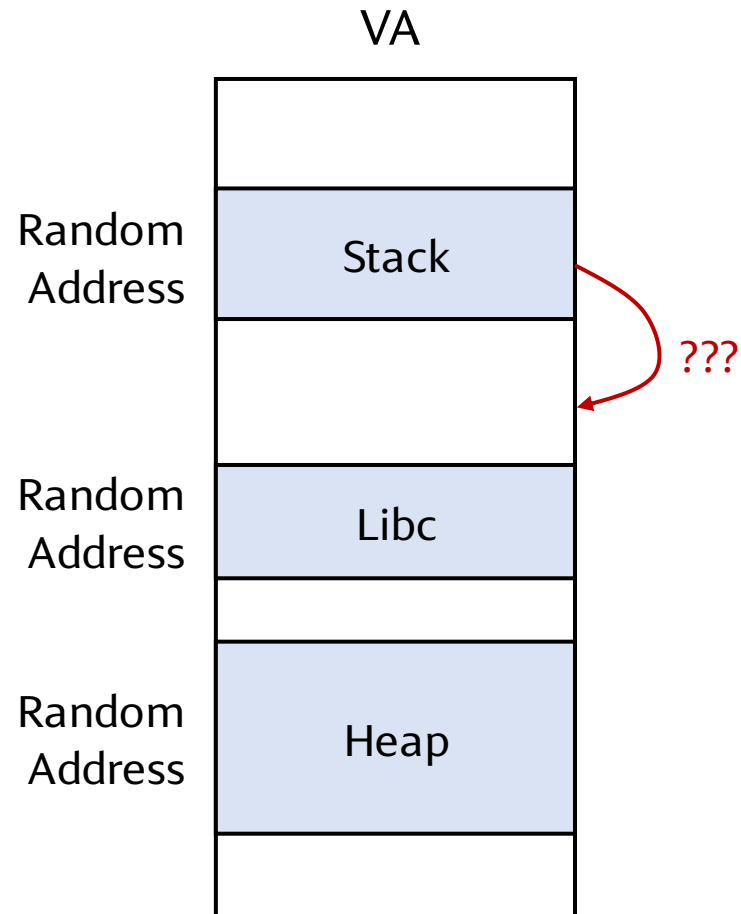
Adapted from “SoK: Eternal War in Memory”¹ (Simplified²)



¹Szekeres et al., “SoK: Eternal War in Memory,” SP’13

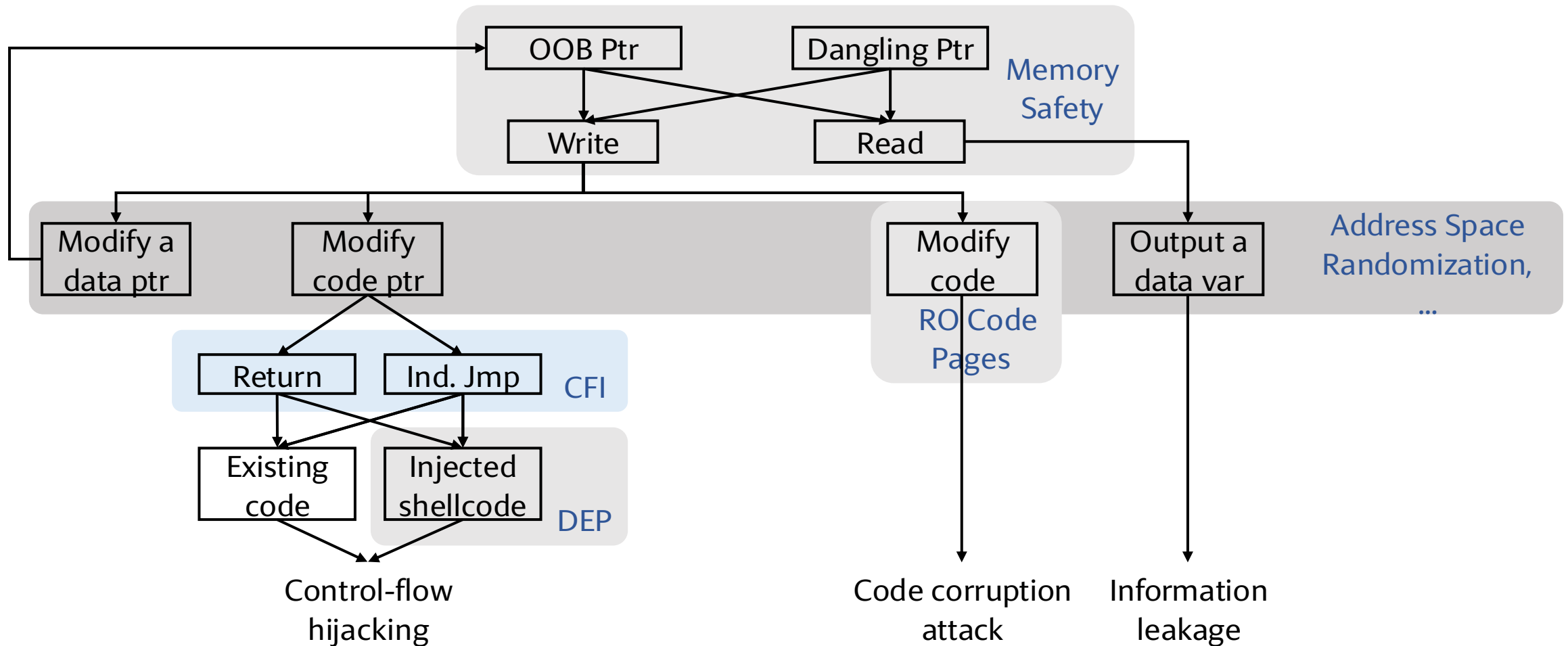
²Only a subset of defenses are listed. Some adaptations are made.

Address Space Layout Randomization (ASLR)



Approach: allocate the stack, heap, and libraries at a random address before each execution

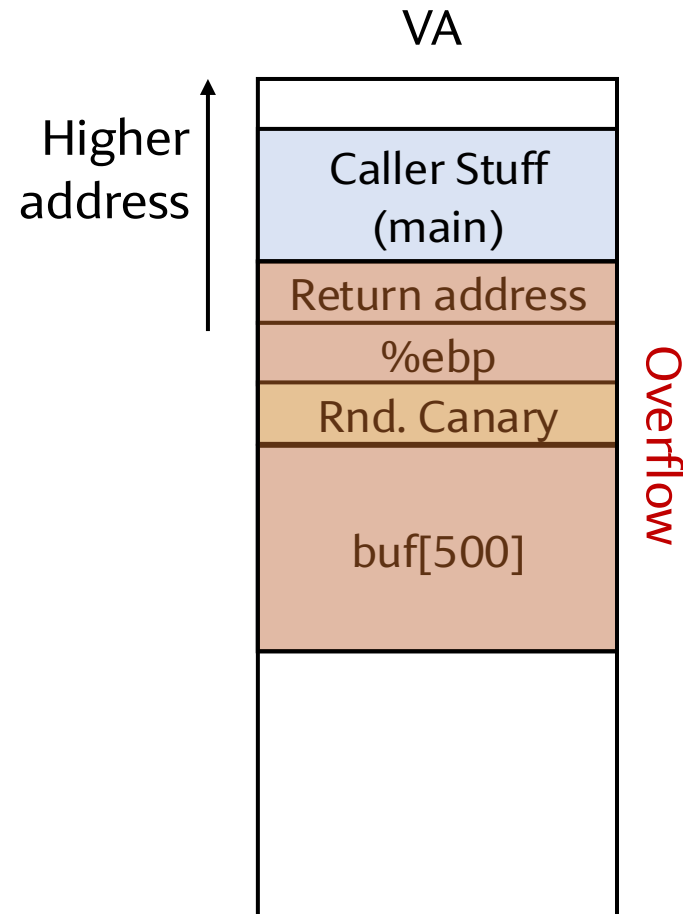
Adapted from “SoK: Eternal War in Memory”¹ (Simplified²)



¹Szekeres et al., “SoK: Eternal War in Memory,” SP’13

²Only a subset of defenses are listed. Some adaptations are made.

Detecting Stack Corruption with Stack Canary

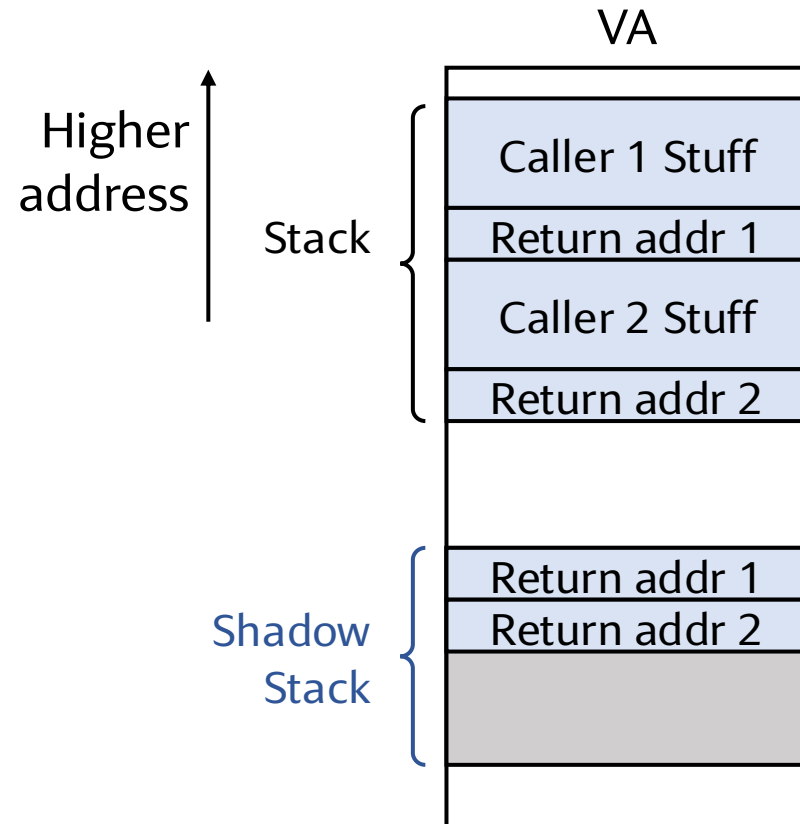


Approach: put a random (64-bit) canary value between the return address & %ebp and the current stack frame
⇒ Check if the canary is modified before returning

The canary is a global random constant for every process or thread

Backward-Edge Control-Flow Integrity (CFI)

Hardware support: Intel Control-flow Enforcement Technology (CET)



- When executing a “call” instruction, the CPU saves a copy of the return address in a shadow stack
- The shadow stack is NOT writable by normal stores
- When the callee returns, the CPU pops the saved address from the shadow stack and compares it against the address popped from the stack
 - Mismatch $\Rightarrow$ CPU raises a fault

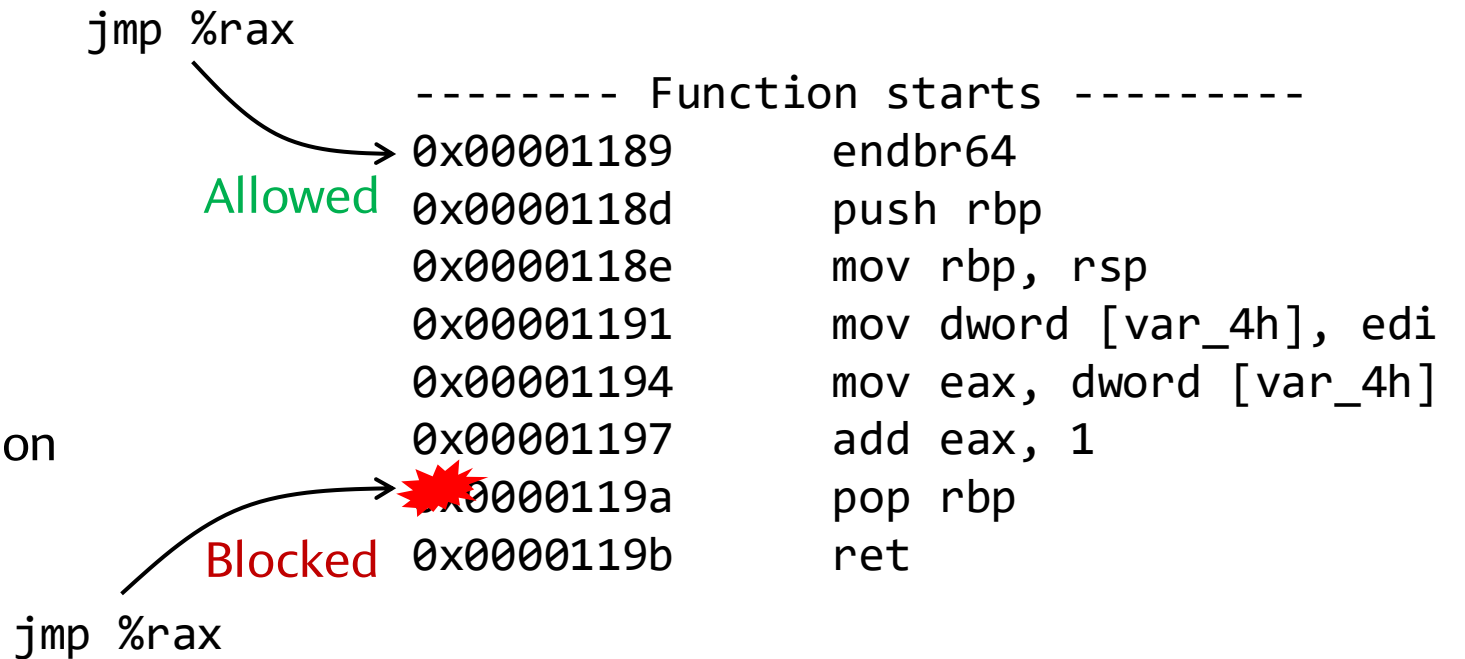
What about indirect jumps/calls?

Forward-Edge Control-Flow Integrity (CFI)

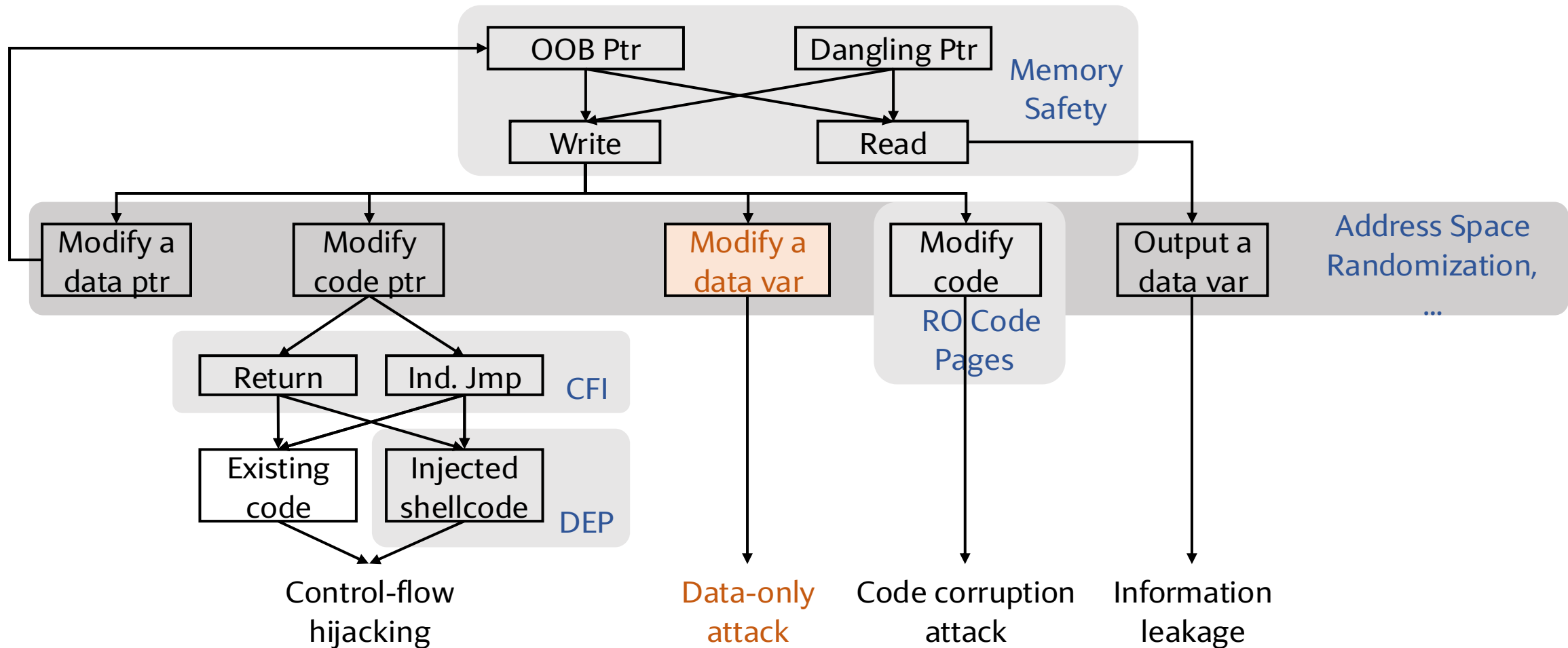
Hardware support: Intel Control-flow Enforcement Technology (CET)

Indirect Branch Tracking:

- New instruction “endbr”
 - 32-bit: endbr32
 - 64-bit: endbr64
- Indirect jumps and calls must land on an “endbr” instruction



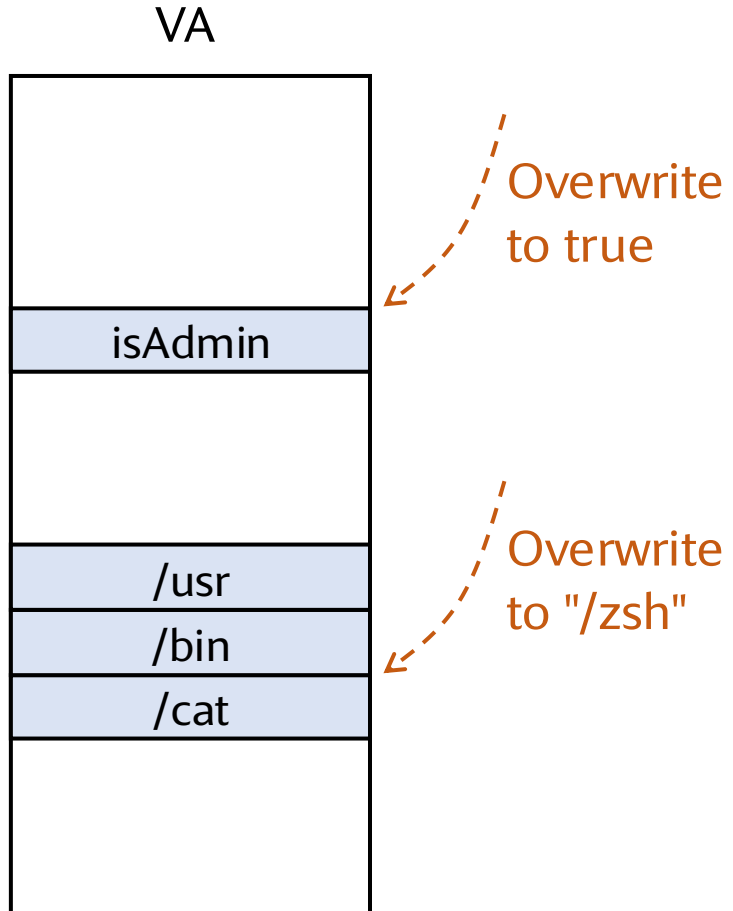
Adapted from “SoK: Eternal War in Memory”¹ (Simplified²)



¹Szekeres et al., “SoK: Eternal War in Memory,” SP’13

²Only a subset of defenses are listed. Some adaptations are made.

Data-Only Attacks

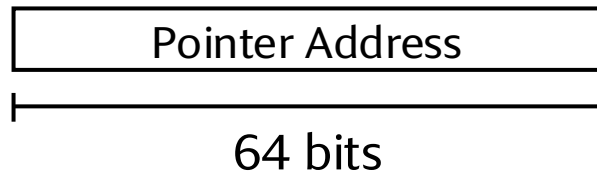


```
isAdmin = false;  
// ...  
if (isAdmin) {  
    // privileged operations  
}
```

```
char *ptr = ...; // Points to "/usr/..."  
system(ptr); // Launches cat
```

CHERI: Capability Hardware Enhanced RISC Instructions

General Purpose Register

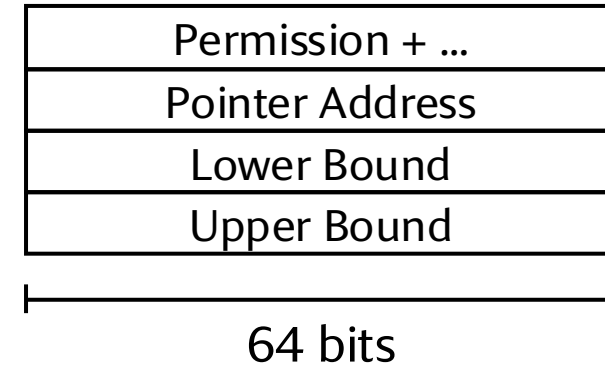


```
char buf[10]; // [0x100, 0x10a)
r10 = buf + 0x20;

*r10 = 0xaa;
```

Out-of-bound write succeeds

Capability Register (256-bit)

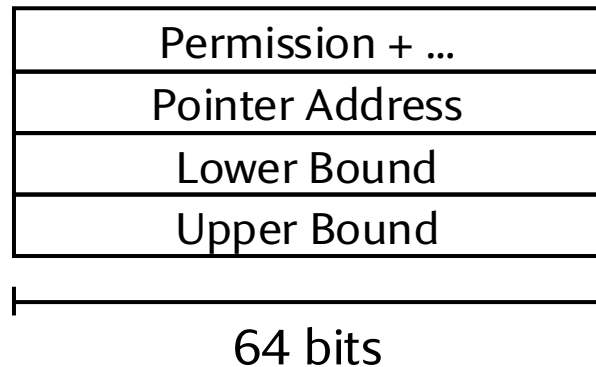


```
char buf[10]; // [0x100, 0x10a)
cr10 = buf + 0x20;
// Compiler emits code to set cr10:
// vaddr=0x120, lb=0x100, ub=0x10a
*cr10 = 0xaa;
```

Hardware catches the out-of-bound write
=> Hardware fault

Pointer Arithmetic

Capability Register (256-bit)



```
char buf[10];    // [0x100, 0x10a)
```

```
cr1 = buf;       // vaddr=0x100, lb=0x100, ub=0x10a
```

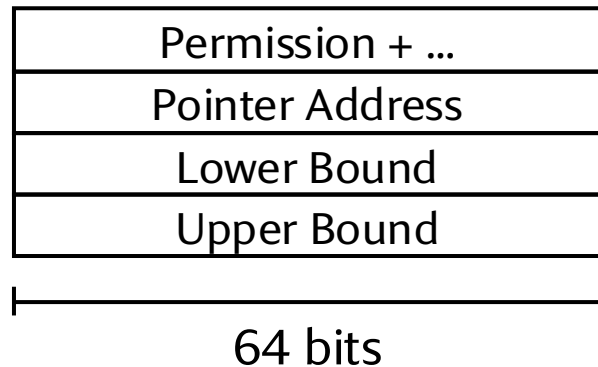
```
cr2 = cr1 + 6;   // vaddr=0x106, lb=0x100, ub=0x10a
```

```
cr3 = cr2 + 10;  // vaddr=0x110, lb=0x100, ub=0x10a  
*cr3 = 100;     // fault!
```

CHERI allows an out-of-bound pointer; it only faults when it is dereferenced

Deriving New Capabilities

Capability Register (256-bit)



```
char buf[256]; // [0x100, 0x200)
```

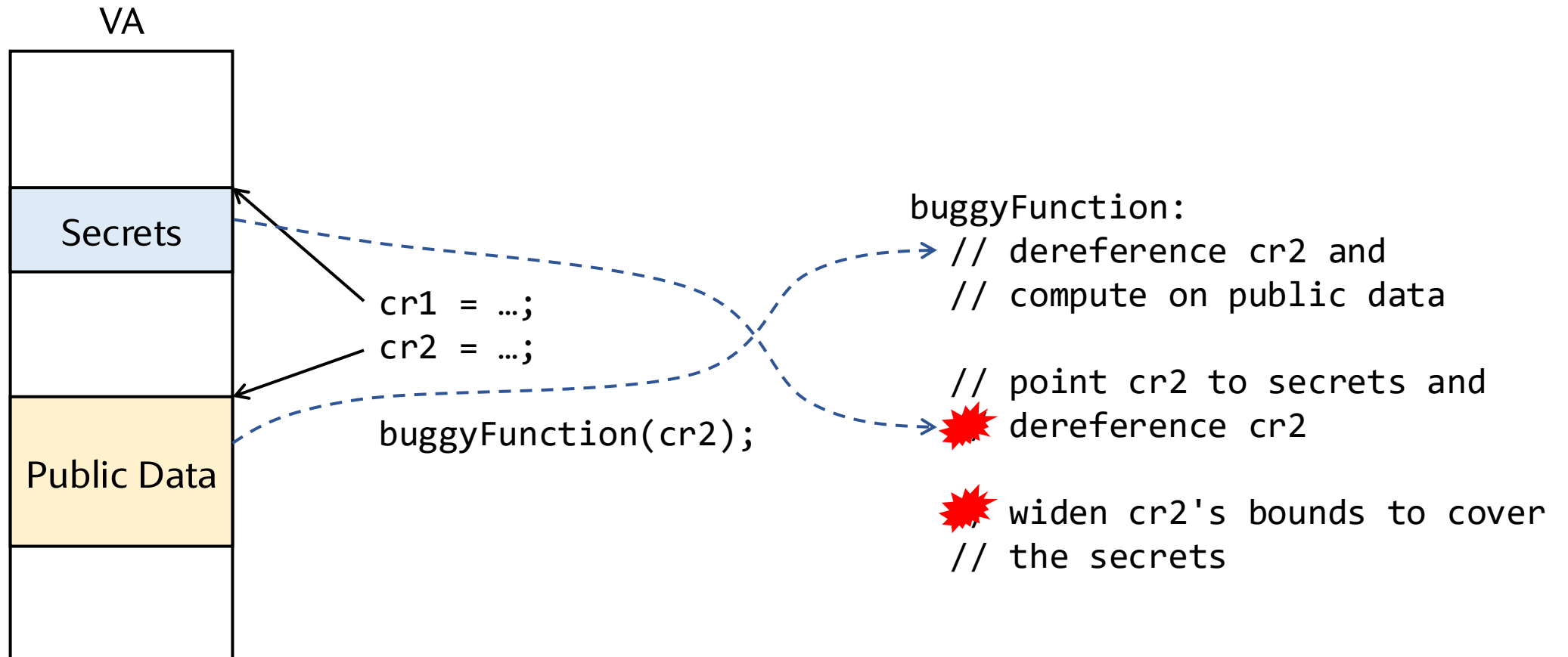
```
cr1 = &buf[128]; // vaddr=0x180, lb=0x100, ub=0x200
```

```
inc_base(cr1, 0x80); // vaddr=0x180, lb=0x180, ub=0x200  
reduce_len(cr1, 0x40); // vaddr=0x180, lb=0x180, ub=0x1c0  
set_perm(cr1, RD_ONLY);
```

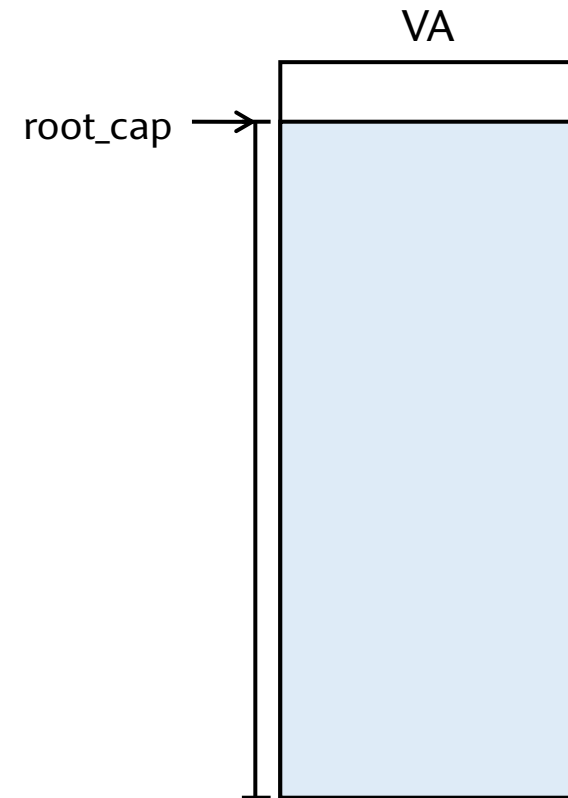
```
// cr1 can only read a subset of the array
```

New capabilities can become stricter, but not more permissive

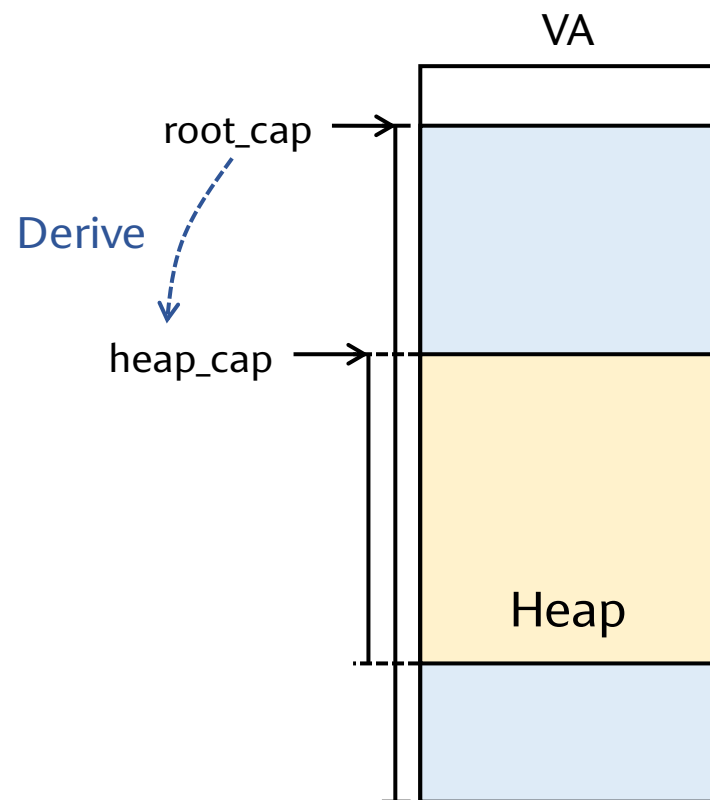
Capability Monotonicity



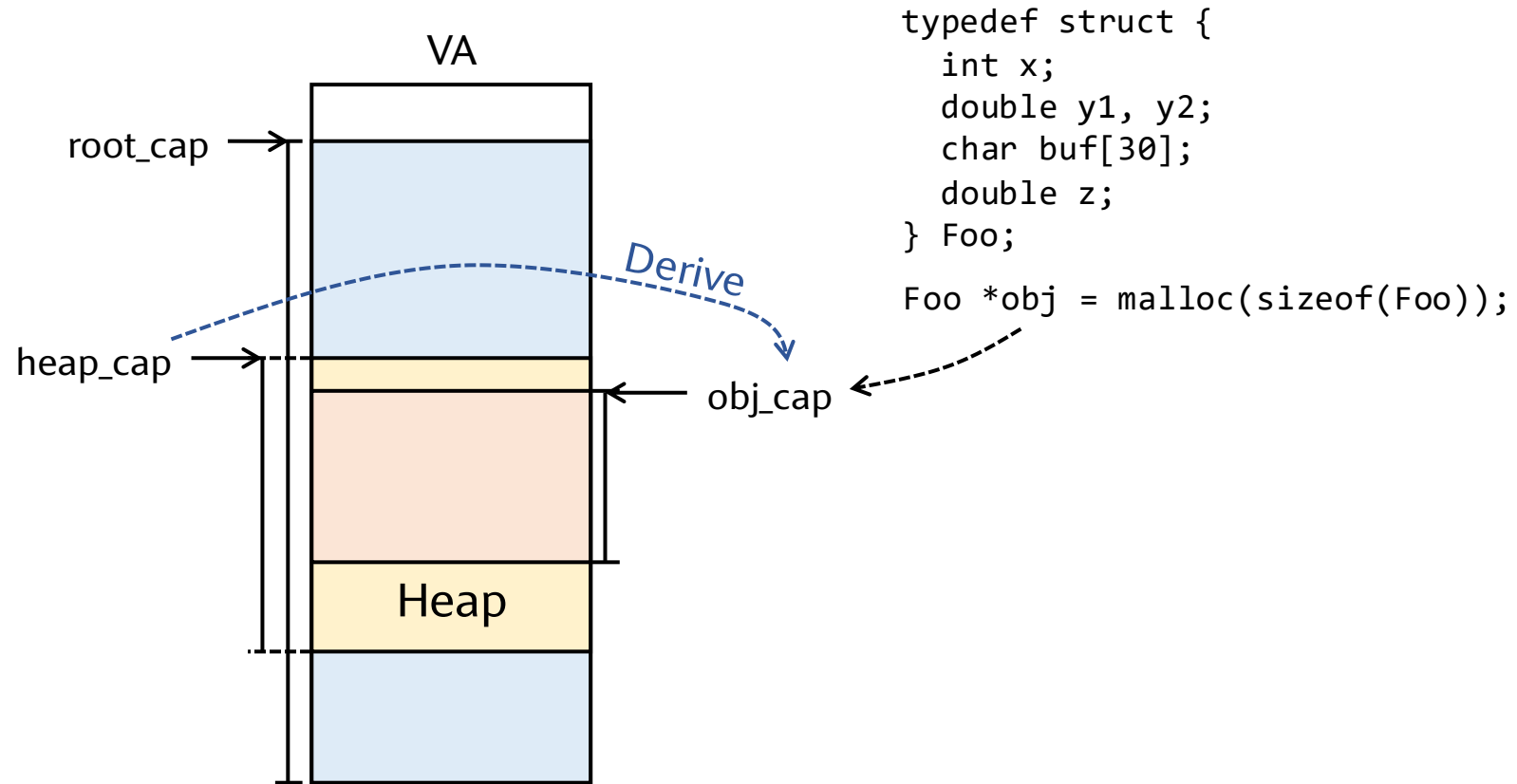
Capability Hierarchy



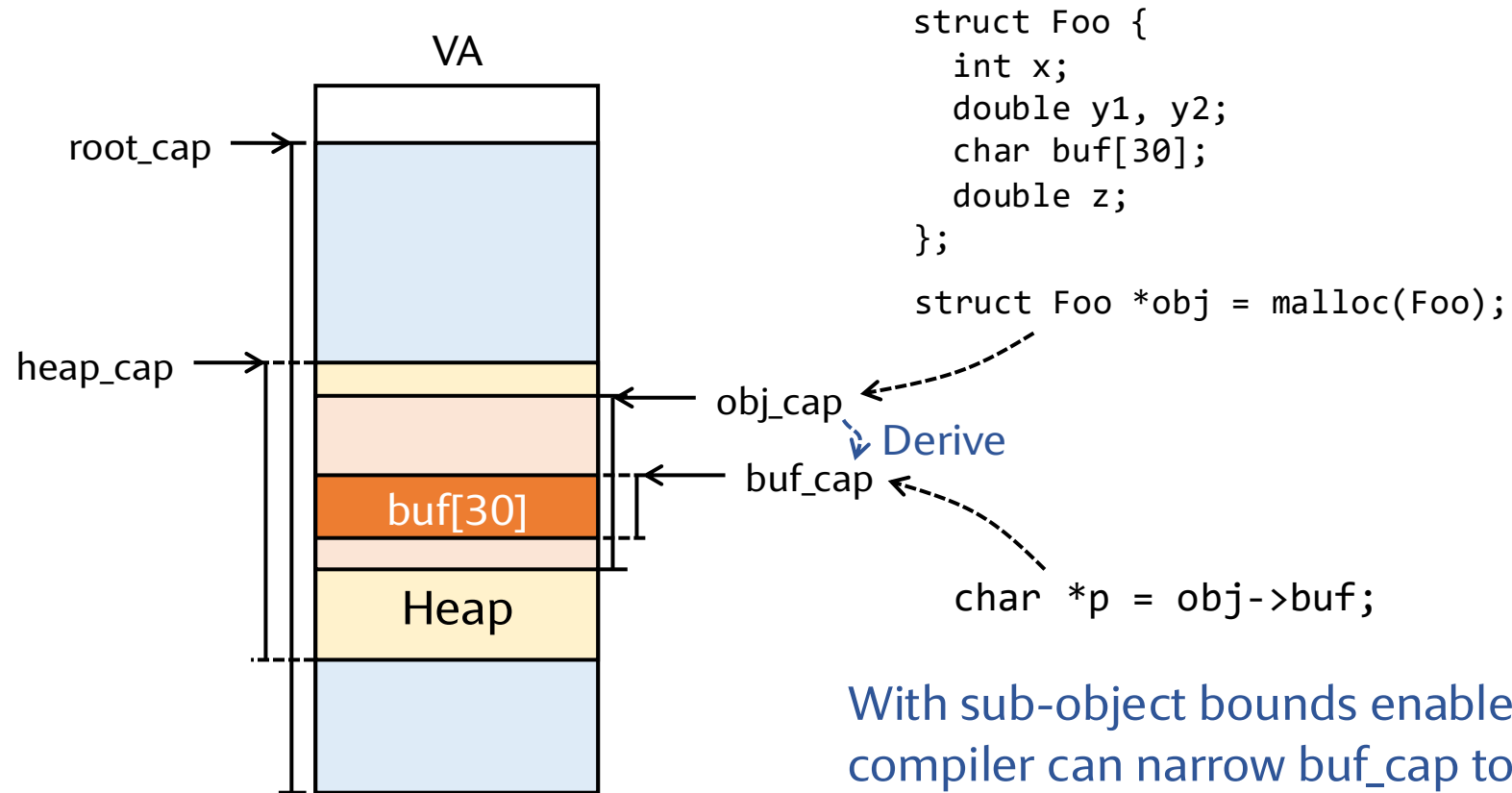
Capability Hierarchy



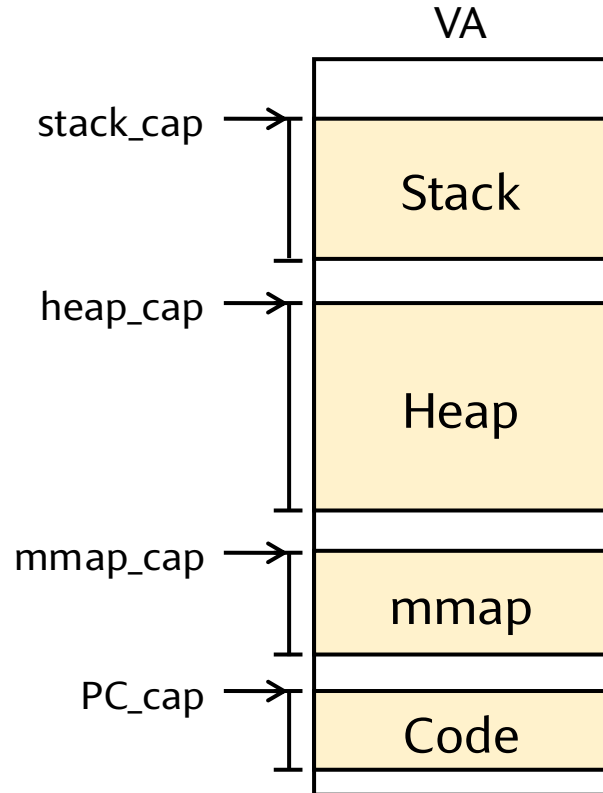
Capability Hierarchy



Capability Hierarchy



Capability Hierarchy

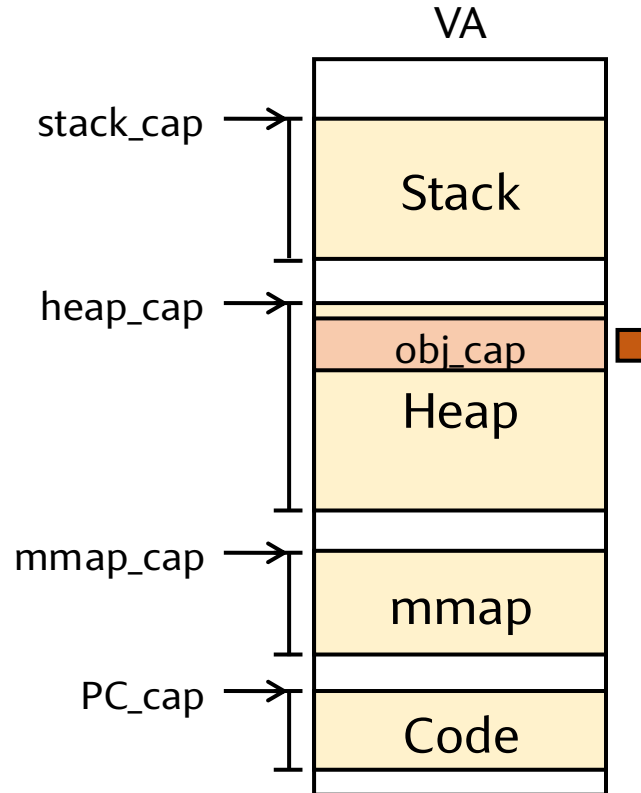


Similar capability hierarchies exist for other memory regions

Generally, the compiler and memory allocator are responsible for correctly deriving child capabilities

⇒ **Require recompiling the program**

Safely Storing Capabilities in Memory



Question: What can go wrong?

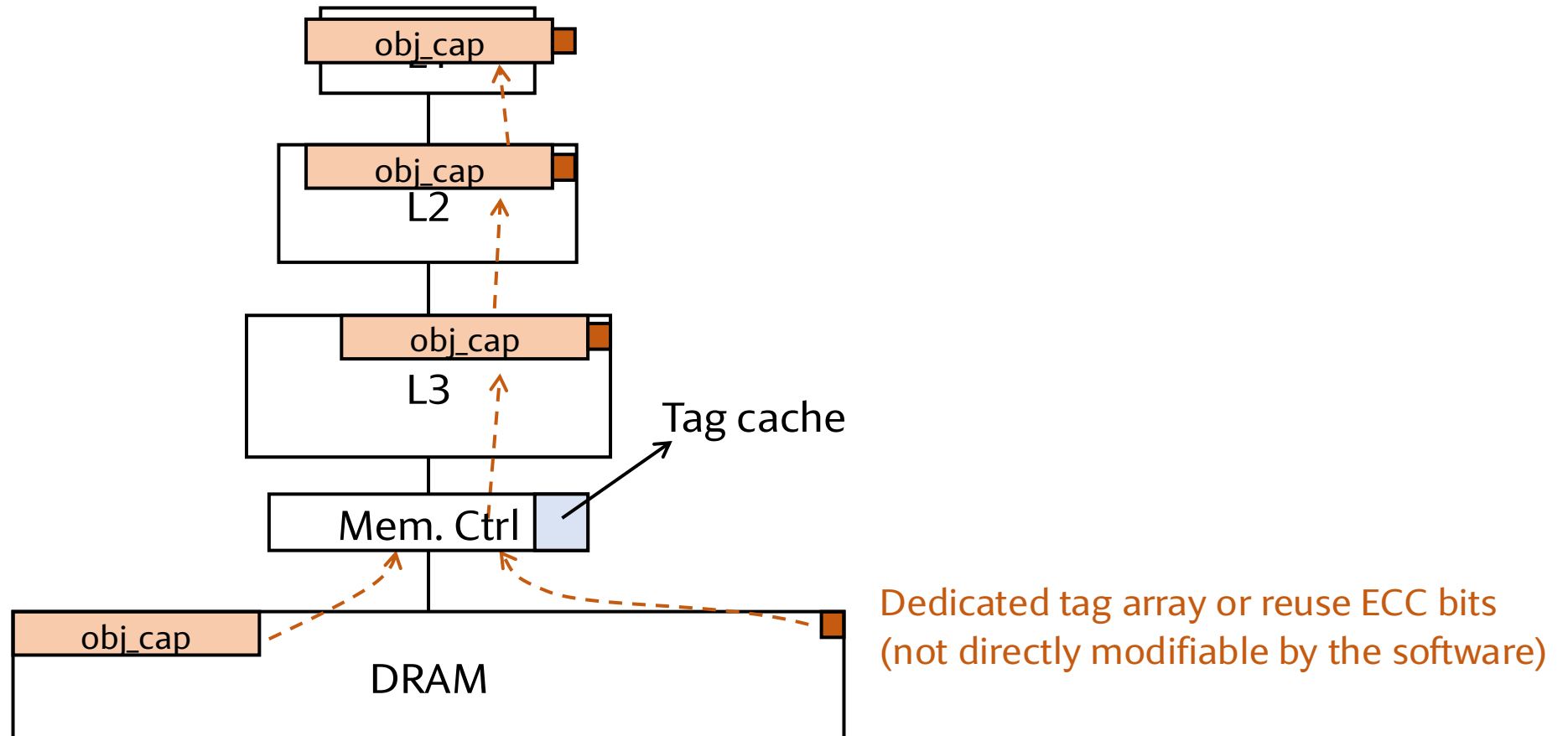
```
heap_cap.addr = &obj_cap;  
heap_cap->lb = 0x0;  
heap_cap->ub = 0xffffffffffffffffffff;
```

Solution: Each capability has a valid bit, which is stored in a location cannot be modified by the software

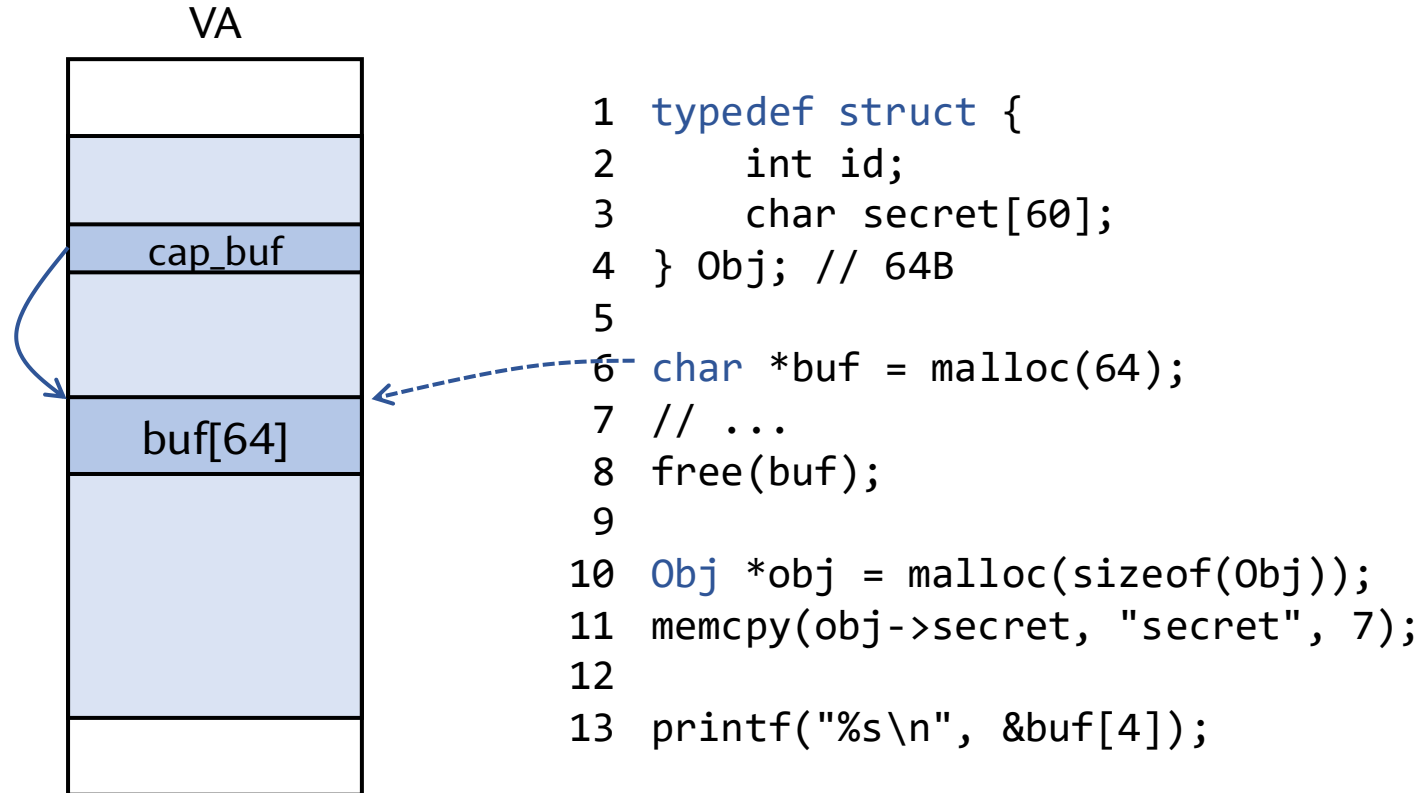
- Dedicated instructions for storing/loading capabilities
- Normal data stores to capabilities **invalidates** those capabilities

⇒ Prevent forgery

Safely Storing Capabilities in Memory

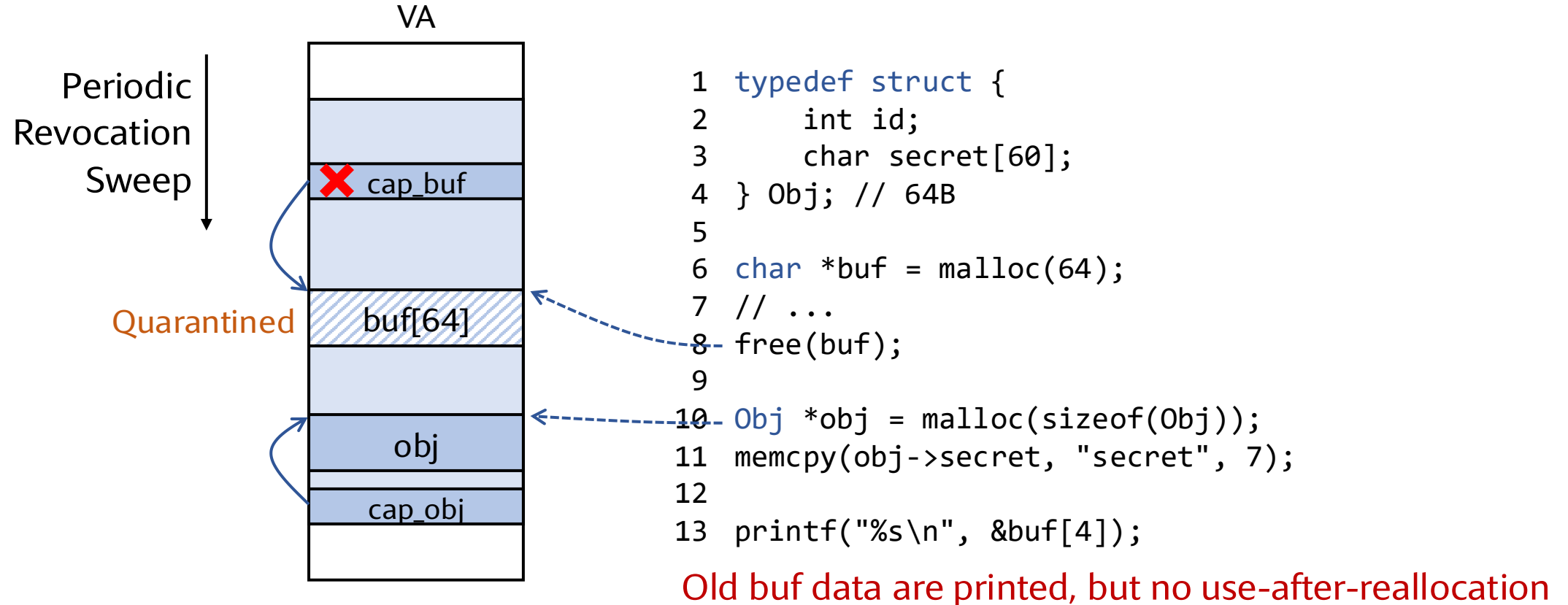


Guaranteeing Temporal Memory Safety*



*Filardo et al., "Cornucopia Reloaded: Load Barriers for CHERI Heap Temporal Safety," ASPLOS '24

Guaranteeing Temporal Memory Safety*



*Filardo et al., "Cornucopia Reloaded: Load Barriers for CHERI Heap Temporal Safety," ASPLOS '24

Compact Capability Representation

Capability Register (256-bit)

Permission + ...
Pointer Address
Lower Bound
Upper Bound

64 bits

0x1234abcd00
0x1234aba000
0x1234abf000

Common prefix,
much redundancy

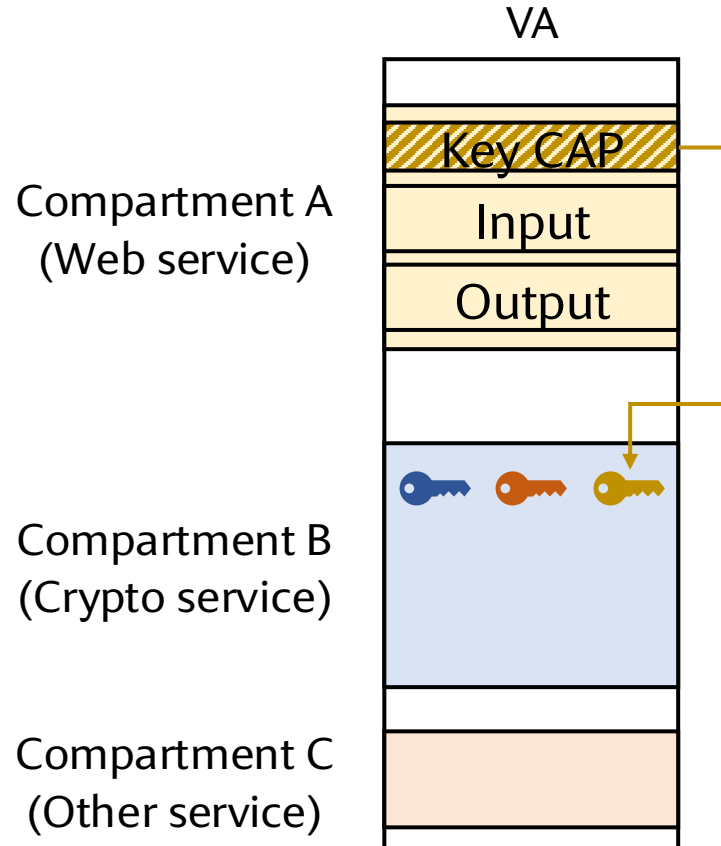


Compact Representation (128-bit)

Bounds	Perm.	Type
Pointer Address		

64 bits

Compartmentalization and Sealed Capability



Goal:

- The web service is allowed to invoke the crypto service to encrypt a given input using the yellow key
- The yellow key is exclusive to the web service
- We do not want to reveal the raw key bytes

Sealed Capability: Still stores the pointer, bounds, and permission information. But it cannot be dereferenced
⇒ **Opaque handle**

Process:

- Web service invokes the crypto service, passing the **sealed** yellow key capability and caps to input/output buffers
- Crypto service checks its validity, **unseals** it, uses it to access the key, reads the input, and writes the ciphertext output

Trusted Domain Switch with Sealed Capability

